

Proyecto1 - Monopoly Distribuido con Estructuras Lineales

Algoritmos y Estructuras de datos I

Profesor: Santiago Gamboa Ramirez

Estudiantes:

Santiago Andrés Allon Quesada - 2026099579

Mathias Araya Gómez - 2026100922

Gerald Andrés Calderón Navarro - 2026020462

Andrés Felipe Guerrero Cervantes - 2026094459

Semestre II 2026

Repositorio oficial del proyecto en GitHub:

https://github.com/Chanty46/Monopoly_Datos_I

INTRODUCCIÓN:

El presente documento describe el diseño y desarrollo del proyecto Monopoly Distribuido con Estructuras Lineales, el cual consiste en una versión simplificada de Monopoly Electrónico. El objetivo principal de este sistema no es recrear la totalidad de las reglas del juego tradicional, sino utilizarlo como un contexto práctico para aplicar correctamente la programación orientada a objetos y el uso de estructuras de datos lineales dentro de una aplicación real.

El sistema está diseñado bajo una arquitectura cliente-servidor comunicada mediante sockets TCP. En esta estructura, el servidor (o banco) opera de manera automática para mantener centralizado y validar el estado oficial de la partida —incluyendo saldos, posiciones, propiedades, turnos y transacciones—, impidiendo que los clientes modifiquen directamente esta información. La aplicación soporta partidas para cuatro jugadores y requiere el uso de al menos dos computadoras físicas conectadas en red.

Adicionalmente, el proyecto integra un módulo de hardware físico (cajero electrónico) implementado con un lector de tarjetas RFID y dados electrónicos. Este componente permite que cada jugador se identifique mediante una tarjeta que simula una billetera electrónica, combinando el desarrollo de software distribuido con la integración de sistemas embebidos y dispositivos electrónicos.

DESARROLLO:

-Bitácora

16 de septiembre

-Santiago: Se inició el desarrollo del proyecto. Durante este primer día se comenzaron a definir las bases sobre las cuales se construiría el juego, estableciendo las primeras clases y estructuras necesarias para manejar la lógica principal. Este trabajo inicial permitió tener una estructura más clara del proyecto y sentó las bases para poder ir agregando posteriormente las diferentes funcionalidades del juego. (Commit inicial en GitHub: [**f3cbc73**](#))

18 de septiembre

-Santiago: Se continuó con el desarrollo de la lógica principal del juego, incorporando diferentes elementos necesarios para su funcionamiento, como el tablero, los turnos y las primeras funcionalidades relacionadas con las casillas. También se realizaron avances iniciales para comprobar el funcionamiento de los dados y de los turnos de forma simulada, con el objetivo de verificar que la lógica básica estuviera funcionando correctamente antes de continuar agregando más funcionalidades. Además, se comenzó a trabajar en la interfaz del proyecto para ir conectando posteriormente la parte visual con la lógica desarrollada. (Commit: [**8f3154f**](#))

-Mathias: Se establecieron las bases de la interfaz gráfica del proyecto, comenzando a preparar la parte visual que permitiría interactuar con el juego. También se integraron los avances que ya se habían realizado en la rama de desarrollo del juego, de manera que la interfaz pudiera comenzar a trabajar sobre la lógica que ya estaba siendo implementada. Esto permitió empezar a unir el trabajo realizado en las diferentes partes del proyecto. (Commit: [**bdd1577**](#))

20 de septiembre

-Gerald: Se implementó y desarrolló la lógica relacionada con los turnos del juego. Se trabajó en el funcionamiento necesario para controlar cuál jugador debía jugar en cada momento y en cómo se debía manejar el cambio entre los diferentes turnos. Esta parte fue importante para establecer un flujo de juego más ordenado y permitir que las acciones de cada jugador se realizarán en el momento correspondiente. (Commit: [**f0366d7**](#))

21 de septiembre

-Gerald: Se agregó la lógica necesaria para el manejo de las cartas de evento del juego. Para administrar estas cartas se implementó una lista circular, permitiendo organizar las cartas de manera que el recorrido por el mazo pudiera continuar de forma ordenada y regresar nuevamente al inicio cuando fuera necesario. Con esto se comenzó a establecer una estructura que permitiera utilizar las cartas de evento durante el desarrollo de las partidas. (Commit: [24603f2](#))

22 de septiembre

-Gerald: Se crearon las primeras cartas que formarían parte del mazo de eventos. Estas cartas fueron necesarias para comenzar a darle contenido y funcionalidad al sistema de eventos que se había implementado anteriormente. De esta manera, la estructura de cartas ya no solo estaba preparada a nivel de código, sino que comenzó a contar con los diferentes eventos que posteriormente podrían aparecer durante una partida. (Commit: [471d414](#))

-Felipe: Se inició el desarrollo del circuito y estudio del datasheet de los componentes, procurando tener buen manejo de cables y legibilidad del circuito, para evitar complicaciones posteriores en caso de necesidad de corrección en algún aspecto.

23 de septiembre

-Santiago: Se realizaron diferentes refactorizaciones en los métodos relacionados con la lógica del juego, buscando mejorar y ordenar el código que ya se había desarrollado. Entre estos cambios se trabajó en el procesamiento de las casillas y en el movimiento de los jugadores mediante los dados, revisando que estos métodos funcionan de la manera esperada. Además, se inició la implementación del banco, que sería otra de las partes importantes para el funcionamiento del juego. También se continuó con la definición de las propiedades de las diferentes clases y se revisó el funcionamiento de los métodos existentes para detectar posibles problemas y dejar una base más estable para continuar con el desarrollo. (Commit: [3532636](#))

24 de septiembre

-Felipe: Se implementó la comunicación entre el servidor y el cliente, incorporando la estructura necesaria para que el proyecto pudiera funcionar mediante sockets TCP. Se trabajó en la conexión necesaria para permitir el intercambio de información entre ambas partes del sistema. Posteriormente, estos cambios fueron integrados al desarrollo principal del proyecto, permitiendo que la parte de comunicación comenzará a formar parte de la estructura general del juego. (Commit: [4a5c697](#))

27 de septiembre

-Gerald: Se implementó la casilla de evento, permitiendo incorporar este tipo de casilla dentro del funcionamiento general del tablero. Además, se agregó el registro y procesamiento de los turnos y jugadores, con el objetivo de llevar un mejor control de las acciones realizadas durante la partida. Esto permitió continuar conectando la lógica de los jugadores y los turnos con los diferentes elementos que forman parte del tablero. (Commit: [a2b8148](#))

-Santiago: Se finalizó y revisó la funcionalidad relacionada con las casillas, comprobando que las diferentes acciones asociadas a ellas funcionaran correctamente. También se implementó el banco y se desarrolló un juego local, con el objetivo de poder probar diferentes partes del juego sin depender todavía de la conexión entre varios dispositivos. Además, se recuperó y continuó el desarrollo de los inputs del juego local después del inconveniente ocurrido con GitHub. Finalmente, se integraron los cambios realizados en las diferentes partes del desarrollo para mantener el proyecto actualizado y juntar los avances realizados por el equipo. (Commit: [3965954](#))

-Felipe: Se inició el armado del circuito correspondiente al proyecto. Durante este proceso se fueron corrigiendo algunos detalles que aparecieron mientras se realizaba el montaje y se llevaron a cabo diferentes pruebas técnicas para verificar que los componentes estuvieran funcionando correctamente. Estas primeras pruebas permitieron identificar posibles problemas y realizar ajustes antes de completar el circuito.

28 de septiembre

-Mathias: Se inició la elaboración de la documentación del proyecto. Se comenzó a organizar y redactar la información necesaria para explicar el funcionamiento y desarrollo del proyecto, avanzando de manera considerable hasta dejarla prácticamente finalizada. Este trabajo permitió empezar a dejar registrado de forma más ordenada todo lo realizado durante el desarrollo del sistema.

-Felipe: Se completó el armado del circuito, con excepción de la sección correspondiente al display de 7 segmentos, debido a la falta de algunas resistencias necesarias para completar esa parte. Durante las pruebas realizadas también se identificó un problema de hardware relacionado con el RFID, ya que el componente no respondió correctamente después de varias pruebas. A partir de estas pruebas se identificaron los pines SCK y MOSI como posibles puntos donde podía encontrarse el problema. Además, se agregó el código

correspondiente al RASP y el controlador, continuando con la integración entre la parte física y el software del proyecto. (Commit: [274838c](#))

29 de septiembre

-Felipe: Se finalizó el circuito, realizando nuevamente las pruebas correspondientes y utilizando un nuevo RFID, con el cual se obtuvieron resultados satisfactorios. También se realizaron las pruebas de los displays de 7 segmentos para comprobar su funcionamiento. Por otra parte, se corrigieron la mayoría de los errores de incompatibilidad que se habían presentado en GitHub, realizando los merges correspondientes en las diferentes ramas para integrar los avances y dejar el proyecto más unificado. (Commit: [ba6f956](#))

-Mathias: Se trabajó en el tablero para lograr que este pudiera visualizarse de manera gráfica dentro de la interfaz del proyecto. Los cambios se mantuvieron sin subir temporalmente debido a que se estaba a la espera de corregir un error relacionado con Casilla.cs y Program.cs, ya que era necesario solucionar este inconveniente antes de integrar completamente los cambios. Además, se continuó trabajando en la documentación del proyecto, avanzando con la información y explicaciones necesarias.

-Gerald: Se realizó todo lo relacionado con el socket, trabajando en la parte de comunicación necesaria para el funcionamiento del proyecto. Al finalizar el trabajo, esta parte quedó lista para realizar las pruebas en conjunto al día siguiente, permitiendo comprobar la comunicación entre las diferentes partes del sistema de manera integrada. (Commit: [daafac4](#))

30 de septiembre

-Todos: Nos reunimos en el apartamento de Felipe para trabajar en conjunto en la corrección de los diferentes errores que se habían presentado en GitHub y poder integrar correctamente los avances realizados por cada integrante. Durante la reunión también se realizaron pruebas del circuito, comprobando que la parte física estuviera funcionando correctamente. Además, se probó la conexión entre el cliente y el servidor mediante los sockets y se realizaron muchas otras pruebas de las diferentes funcionalidades del proyecto. Esta reunión permitió comprobar de una manera más completa cómo funcionaban las distintas partes al estar integradas y detectar errores que no habían sido evidentes al trabajar cada parte por separado.

-Mathias: Se continuó trabajando en la documentación del proyecto, avanzando aproximadamente hasta un 50% de su finalización. Se siguió organizando la información y

desarrollando las diferentes secciones necesarias para dejar documentado el trabajo realizado durante el desarrollo del proyecto.

-Santiago: Se trabajó en resolver la visualización del display de 7 segmentos, comprobando las pruebas de dígitos y mejorando el código correspondiente. (Commit: [19c3c3f](#))

1 de octubre

-Felipe: Se realizaron correcciones de bugs menores que estaban evitando la compilación correcta del proyecto. También se eliminaron archivos huérfanos y archivos temporales que ya no eran necesarios, con el objetivo de mantener el proyecto más limpio y evitar elementos que pudieran generar problemas durante la compilación o el desarrollo. Además, se separaron formalmente las clases de Banco y Dado, asegurando el soporte para partidas de 4 jugadores. (Commit: [4459f4f](#))

-Mathias: Se finalizó la documentación del proyecto. Para completar el documento se implementó el diagrama de clases, además de redactar la introducción y la conclusión y agregar la explicación de las estructuras utilizadas en el desarrollo. Con esto se completaron las diferentes secciones necesarias y se dio por concluida la documentación del proyecto.

-Gerald: Se implementó la funcionalidad correspondiente a los dados dobles, trabajando en la lógica necesaria para que esta característica funcionara correctamente dentro del juego. Después de realizar las respectivas pruebas y ajustes, los dados dobles quedaron funcionales al 100%, completando así esta parte de la lógica del juego. (Commit: [d7e9d14](#))

Explicación de estructuras utilizadas

Uno de los requisitos más importantes de este proyecto fue no utilizar listas ya hechas de C# (como List o LinkedList), sino programar nuestras propias estructuras de datos desde cero usando nodos y punteros enlazados a mano. Cada estructura cumple una función específica dentro de las reglas del Monopoly:

1. El Tablero (Lista Circular Doblemente Enlazada): ¿Cómo funciona? Imagina el tablero como un camino cerrado en círculo con 24 casillas numeradas del 0 al 23. Como la última casilla se conecta de nuevo con la primera (Salida), los jugadores pueden dar vueltas indefinidamente. Además, como está doblemente enlazada, cada casilla sabe quién está adelante y quién está atrás, lo que permite moverse hacia adelante o retroceder si una carta de evento lo indica.

¿Cómo se usa en una partida? Cuando un jugador tira los dados y saca por ejemplo un 5, el método moverJugadorPorDados avanza paso a paso por los nodos. Si en ese camino pisa o cruza la casilla 0 (Salida), el juego detecta el cruce de inmediato y le regala \$200 de bono.

Operaciones clave: agregarCasilla (para armar el tablero), buscarCasillaPorID (para encontrar casillas fijas como la cárcel) y moverJugadorPorDados (para avanzar contando los pasos).

2. La Rotación de Turnos (Lista Circular Simplemente Enlazada): ¿Cómo funciona? Funciona exactamente como una ronda de amigos donde cada uno tiene su turno en orden (Jugador 1, luego 2, luego 3, luego 4 y vuelve a empezar con el 1). Cada vez que la ronda vuelve al inicio, el sistema suma una nueva ronda de juego.

¿Qué pasa si alguien pierde? Si un jugador se queda sin dinero y cae en bancarrota, el método avanzarTurno lo detecta mediante una bandera (isActive) y simplemente lo salta para darle el turno al siguiente participante activo, asegurando que el juego nunca se detenga.

Operaciones clave: agregarJugador (registra a los participantes), avanzarTurno (pasa la bola al siguiente jugador solvente) y CantidadJugadoresActivos (cuenta cuántos jugadores siguen vivos).

3. La Cartera de Propiedades (Lista Lineal Simplemente Enlazada): ¿Cómo funciona? Cada jugador tiene su propia lista privada donde guarda los títulos de las propiedades que va comprando durante la partida.

¿Cómo se usa en una partida? Cuando un jugador compra una casilla libre, esa propiedad

se inserta en su lista personal. Cuando otro jugador cae en esa casilla, el sistema revisa esta lista para saber quién es el dueño y cuánto cobrarle de alquiler. Además, al final de la partida, el método `calcularValorTotal` suma el precio de todas las propiedades del jugador más su dinero en efectivo para conocer su riqueza total (patrimonio).

Operaciones clave: `agregarPropiedad` (añade un nuevo inmueble), `buscarPorId` (revisa si el jugador es dueño de una casilla) y `calcularValorTotal` (suma el valor de todos los bienes).

4. El Historial de Transacciones (Lista Lineal Doblemente Enlazada): ¿Cómo funciona?

Es como el libro de contabilidad oficial del juego, donde se anota cada transacción económica (compras, alquileres, multas y premios). Al ser doblemente enlazada, tiene dos extremos: un puntero al inicio (la primera jugada) y un puntero al final (la última jugada).

¿Por qué es útil el doble enlace? Porque si un jugador quiere ver sus últimos movimientos en la pantalla, no hace falta leer toda la historia desde el principio; el sistema empieza desde el final (Tail) y va retrocediendo hacia atrás usando el puntero Anterior en tiempo récord.

Operaciones clave: `Registrar` (guarda la jugada en memoria y la escribe en `transacciones.txt` al mismo tiempo) y `ObtenerHistorialTexto` (muestra los últimos movimientos hacia atrás).

5. El Mazo de Eventos (Cola Circular Enlazada): ¿Cómo funciona? Modela la baraja de cartas sorpresa del Monopoly (como 'Lotería Nacional: cobra \$100' o 'Impuestos: paga \$50'). Está conectada en círculo permanente, de manera que la última carta apunta de vuelta a la primera.

¿Cómo se usa en una partida? Cuando alguien cae en una casilla de evento, el método `TomarCarta` saca la carta de arriba, aplica el premio o castigo al jugador, y de inmediato la manda al fondo de la baraja. Así, las cartas rotan de forma infinita sin necesidad de barajar ni crear objetos nuevos en memoria.

Operaciones clave: `AgregarCarta` (prepara el mazo al inicio) y `TomarCarta` (saca la carta superior y la rota al fondo).

Explicación de la Distribución del Código y Clases que Existen en el Repositorio

Para que el proyecto sea ordenado, fácil de mantener y comprensible para cualquier persona sin necesidad de conocimientos avanzados, organizamos el código en cuatro áreas claras según la tarea que cumple cada archivo en el sistema:

1. Lógica del Juego y Reglas (Carpeta Monopoly/)

[Tablero.cs](#): Crea el circuito de las 24 casillas oficiales en círculo y mueve las fichas según los dados.

[Casilla.cs](#): Define los diferentes tipos de casillas usando herencia y polimorfismo: propiedades que se pueden comprar, cárcel, policía, salida y eventos.

[Jugador.cs](#): Guarda la información de cada participante: su nombre, saldo en colones, casilla actual y código de tarjeta RFID.

[Turnos.cs](#): Controla a quién le toca jugar en cada momento y cuenta las rondas completadas.

[ListaDePropiedades.cs](#): Administra la cartera de casas y terrenos que cada jugador va adquiriendo.

[Banco.cs](#): Es la entidad financiera oficial del juego; cobra las compras de casillas libres, paga los premios de salida (¢200) y tramita hipotecas al 50% de valor.

[Dado.cs](#): Modela el par de dados (números del 1 al 6) y detecta si salieron números iguales (dados dobles).

[Eventos.cs](#): Maneja las cartas de suerte y sorpresa que premian o cobran dinero a los jugadores.

[Juego.cs](#): La clase que orquesta todo: une el tablero, los turnos y el banco, lleva la cuenta de los turnos globales y decide quién gana la partida.

[Transaccion.cs](#): Registra cada movimiento de dinero en una lista doblemente enlazada y lo guarda en el archivo de texto.

[transacciones.txt](#): El archivo de texto físico donde queda guardado el historial de jugadas para que no se borre al cerrar la partida.

2. Conectividad en Red y Pantallas (Sockets y Web)

[Servidor.cs](#): El cerebro central del juego en red. Escucha en el puerto TCP 5000, recibe a los 4 jugadores y valida cada movimiento para asegurar que nadie haga trampa.

[Cliente.cs](#): El programa de consola que abre cada jugador para conectarse al servidor, tirar los dados y comprar propiedades.

[ServidorWeb.cs](#): Un servidor web embebido (puerto 8080) que muestra una página visual a color con el tablero, los jugadores y las transacciones en tiempo real.

3. Hardware y Microcontrolador (Carpeta Monopoly/RASP/)

[main.py](#): Programa en MicroPython para la Raspberry Pi Pico que atiende los comandos seriales, multiplexa el display de 7 segmentos y lee las tarjetas RFID.

[config_pines.py](#): El mapa único donde se definen a qué patita de la Raspberry Pi Pico va conectado cada cable del circuito.

[ControladorHardware.cs](#): El componente en C# que se comunica por cable USB con la placa. Si la Raspberry Pi no está conectada, activa un modo de respaldo automático para que se pueda jugar con el teclado sin trabarse.

[diag_rfid.py y test_display.py](#): Pequeños programas de prueba para verificar de forma aislada que el lector de tarjetas y la pantalla de números funcionen bien.

[guia_de_uso.md](#): Manual de instrucciones rápidas de instalación y conexión de pines para el hardware.

4. Punto de Inicio y Pruebas Automatizadas

[Program.cs](#): El menú principal de inicio con opciones para levantar el servidor, abrir un cliente o correr pruebas.

[PruebasUnitarias.cs](#): Batería de 123 pruebas automáticas que comprueban que todas las estructuras y reglas funcionen al 100% sin errores.

[PruebaIntegracion.cs](#): Simula una partida completa de prueba entre dos jugadores virtuales para verificar que el sistema responda bien de principio a fin.

[Monopoly.csproj](#): Archivo de configuración del proyecto en .NET 10.

[README.md](#): Página principal de bienvenida del repositorio con el resumen y los enlaces a los entregables.

Protocolo de Comunicación TCP (Sockets en Puerto 5000)

Para conectar a los cuatro jugadores en red usamos sockets TCP en el puerto 5000 bajo el modelo de Servidor Autoritativo. Esto significa que la computadora que actúa como Servidor es el único juez oficial de la partida: los jugadores no pueden cambiarse el dinero ni moverse por sí solos, sino que envían una solicitud (por ejemplo: 'quiero tirar los dados' o 'quiero comprar esta casilla'). El servidor revisa si es su turno y si tiene suficiente dinero; si todo es legal, hace el cambio y le avisa de inmediato a todos los jugadores conectados para que sus pantallas se actualicen.

¿Cómo se envían los mensajes? Son líneas de texto sencillas separadas por el caracter pleca (|) y terminadas con un salto de línea (\n). Por ejemplo: CONECTAR|Santiago o HIPOTECAR|4.

Comandos que el Jugador le Envía al Servidor

Comando	Ejemplo de Mensaje	¿Qué hace el Servidor al recibirlo?
CONECTAR	CONECTAR Santiago	Revisa que haya espacio (máximo 4 jugadores), le da la bienvenida, le asigna su saldo de ₡1500 y le avisa a los demás.
TIRAR_DADOS	TIRAR_DADOS	Revisa que sea el turno de ese jugador y que no haya tirado todavía. Genera los dados, mueve la ficha y avisa a todos.
COMPRAR_PROPIEDAD	COMPRAR_PROPIEDAD	Comprueba que la casilla donde cayó no tenga dueño y que el jugador tenga suficiente dinero. Descuenta el saldo y le entrega la propiedad.
NO_COMPRAR	NO_COMPRAR	El jugador decide no comprar la casilla actual, dejándola disponible para futuros turnos.
TERMINAR_TURNO	TERMINAR_TURNO	Verifica que el jugador ya haya lanzado los dados y le pasa el turno al siguiente jugador activo en la lista circular.
HIPOTECAR	HIPOTECAR 5	Revisa que la propiedad sea del jugador y no tenga casas puestas. El banco le presta el 50% del valor de la casilla.
DESHIPOTECAR	DESHIPOTECAR 5	El dueño paga al banco el 50% más un 10% de interés para volver a activar su propiedad y cobrar alquileres.
CONSULTAR_ESTADO	CONSULTAR_ESTADO	Pide un resumen de cómo va la partida: quién tiene qué casilla, los saldos y a quién le toca jugar.
CONSULTAR_TRANSACCIONES	CONSULTAR_TRANSACCIONES	Pide los últimos movimientos de dinero registrados en el historial.

Respuestas y Avisos que el Servidor Envía a los Jugadores

Mensaje del Servidor	¿A quién le llega?	¿Qué significa?
BIENVENIDO 1 Santiago 1500 24 1 1/4	Al jugador que se conecta	Le confirma que entró con éxito, le da su número de jugador (ID 1) y su saldo de ¢1500.
NUEVO_TURNO 2 Felipe	A todos	Avisa que es el turno del Jugador 2 (Felipe) para que tire los dados.
DADOS_LANZADOS 1 3 2 5 5 Calle Real 1500	A todos	Muestra que el jugador sacó un 3 y un 2 (total 5) y se movió a la casilla 'Calle Real'.
PROPIEDAD_DISPONIBLE 5 Calle Real 120 20	Al jugador en turno	Le avisa que la casilla está libre y le pregunta si desea comprarla por ¢120.
COMPRA_EXITOSA 1 5 Calle Real 1380	A todos	Avisa que el Jugador 1 compró la propiedad y su nuevo saldo es ¢1380.
ALQUILER_PAGADO 2 1 20 1480 1400	A todos	Avisa que el Jugador 2 cayó en la casilla del Jugador 1 y le pagó ¢20 de alquiler.
PARTIDA_FINALIZADA Santiago Mayor patrimonio	A todos	Anuncia el fin de la partida y proclama al ganador.

Validaciones de Reglas y Cómo se Gana la Partida:

Para que el juego sea limpio y divertido, el servidor revisa automáticamente las reglas en cada jugada:

- **Nadie juega fuera de turno:** Si intentas tirar los dados o comprar cuando le toca a otro jugador, el servidor te rechaza amablemente diciendo que no es tu turno.
- **Compras legales:** No puedes comprar una propiedad si ya le pertenece a otro jugador, o si tu saldo en colones no alcanza para pagarla.
- **Hipotecas justas:** Solo puedes hipotecar casillas que sean tuyas y que no tengan construcciones. El banco te presta el 50% de lo que costó. Y si quieres recuperarla, pagas el 50% más un 10% de interés.
- **Regla de los 3 dobles a la cárcel:** Tirar dados dobles (por ejemplo 4 y 4) te da derecho a tirar otra vez. Pero si sacas dobles 3 veces seguidas en la misma ronda, la policía te detiene por sospechoso y vas directo a la Cárcel por 2 turnos.
- **Qué pasa si caes en bancarrota:** Si tienes que pagar un alquiler o una multa y no te alcanza el dinero, entregas todo lo que te queda, tu saldo queda en ₡0 y quedas eliminado de la partida. La ronda de turnos te salta automáticamente.

¿Cómo se Decide al Ganador de la Partida?

El juego evalúa el final de la partida de dos maneras muy claras:

1. Por eliminación (Único Sobreviviente): Si tres jugadores caen en bancarrota y queda una sola persona con dinero en la mesa, esa persona gana automáticamente por ser el único sobreviviente.

2. Por límite de turnos (Mayor Patrimonio): Para que las partidas no se hagan eternas, el juego tiene un límite configurable de turnos (por defecto 100 turnos). Cuando se llega a ese número, el sistema calcula la riqueza total de cada participante activo aplicando la fórmula oficial:

$$\text{Riqueza Total (Patrimonio)} = \text{Dinero en Efectivo} + \text{Valor de Compra de todas tus Propiedades}$$

El jugador que haya acumulado más riqueza sumando su dinero y sus propiedades es declarado el ganador indiscutible.

Diagrama de clases

2,2026-09-30 17:01:03,1,CONEXION,SISTEMA,Jugador 2,1500,"Jugador Jugador 2 (ID 2) ingresó vía Web

con ¢1500."

3,2026-09-30 17:01:23,1,COMPRA,Jugador_1,BANCO,120,"Jugador_1 compró Calle Real por ¢120."

4,2026-09-30 17:01:33,1,EVENTO,BANCO,Jugador_2,100,"Lotería Nacional: ¡Ganaste la lotería semanal! Cobra ¢100."

5,2026-09-30 17:01:49,2,COMPRA,Jugador_1,BANCO,280,"Jugador_1 compró Escazú por ¢280."

6,2026-09-30 17:02:56,2,COMPRA,Jugador_2,BANCO,260,"Jugador_2 compró Curridabat por ¢260."

7,2026-09-30 17:03:04,3,SALIDA,BANCO,Jugador_1,200,"Jugador_1 cayó en la Salida y cobró ¢200."

12,2026-09-30 17:04:20,4,ALQUILER,Jugador_2,Jugador_1,20,"Jugador_2 pagó ¢20 de alquiler en Calle Real a Jugador_1."

Manual Breve de Ejecución (Comandos Reales)

Para correr el juego solo necesitas tener instalado .NET 10 en tu computadora y abrir una terminal en la carpeta del proyecto. Estos son los comandos básicos:

1. Compilar el proyecto: `dotnet build Monopoly/Monopoly.csproj`

2. Iniciar el Servidor (crea la partida y la página web en puerto 8080): `dotnet run --project Monopoly/Monopoly.csproj -- --server 5000`

3. Conectar un jugador por consola: `dotnet run --project Monopoly/Monopoly.csproj -- --client 127.0.0.1 5000 Santiago`
(En otra computadora cambias 127.0.0.1 por la dirección IP de la computadora del servidor)

4. Ver el tablero gráfico en el navegador: Abre tu navegador de internet y escribe: `http://localhost:8080` (o la IP del servidor en el puerto 8080).

5. Correr las pruebas automáticas: `dotnet run --project Monopoly/Monopoly.csproj -- --unit-tests`
(Ejecuta las 123 pruebas unitarias y te confirma que todas pasaron al 100%)

6. Correr la demostración automática de juego: `dotnet run --project Monopoly/Monopoly.csproj -- --integration-test`
(Juega una partida de prueba automática entre dos jugadores simulados)

CONCLUSIONES:

El desarrollo del proyecto Monopoly Distribuido con Estructuras Lineales permitió consolidar de manera práctica los conceptos fundamentales de la ingeniería de software, la programación orientada a objetos y la implementación de estructuras de datos lineales dentro de una aplicación funcional. A través de una arquitectura cliente-servidor basada en sockets TCP, se logró implementar un servidor centralizado encargado de validar y administrar de forma segura y autónoma el estado de la partida, los turnos y las transacciones, asegurando que los clientes no modifiquen directamente la información oficial.

Asimismo, la integración del módulo de hardware electrónico mediante lectores RFID y dados electrónicos enriqueció significativamente el proyecto al fusionar el desarrollo de software en red con los sistemas embebidos, simulando de forma interactiva una billetera electrónica para cada jugador. En definitiva, la ejecución de esta iniciativa cumplió con el propósito de utilizar un juego como contexto analítico para aplicar estructuras de datos complejas, fomentar el trabajo en equipo mediante repositorios y controles de versiones, y cumplir con rigurosos estándares de documentación y defensa técnica.